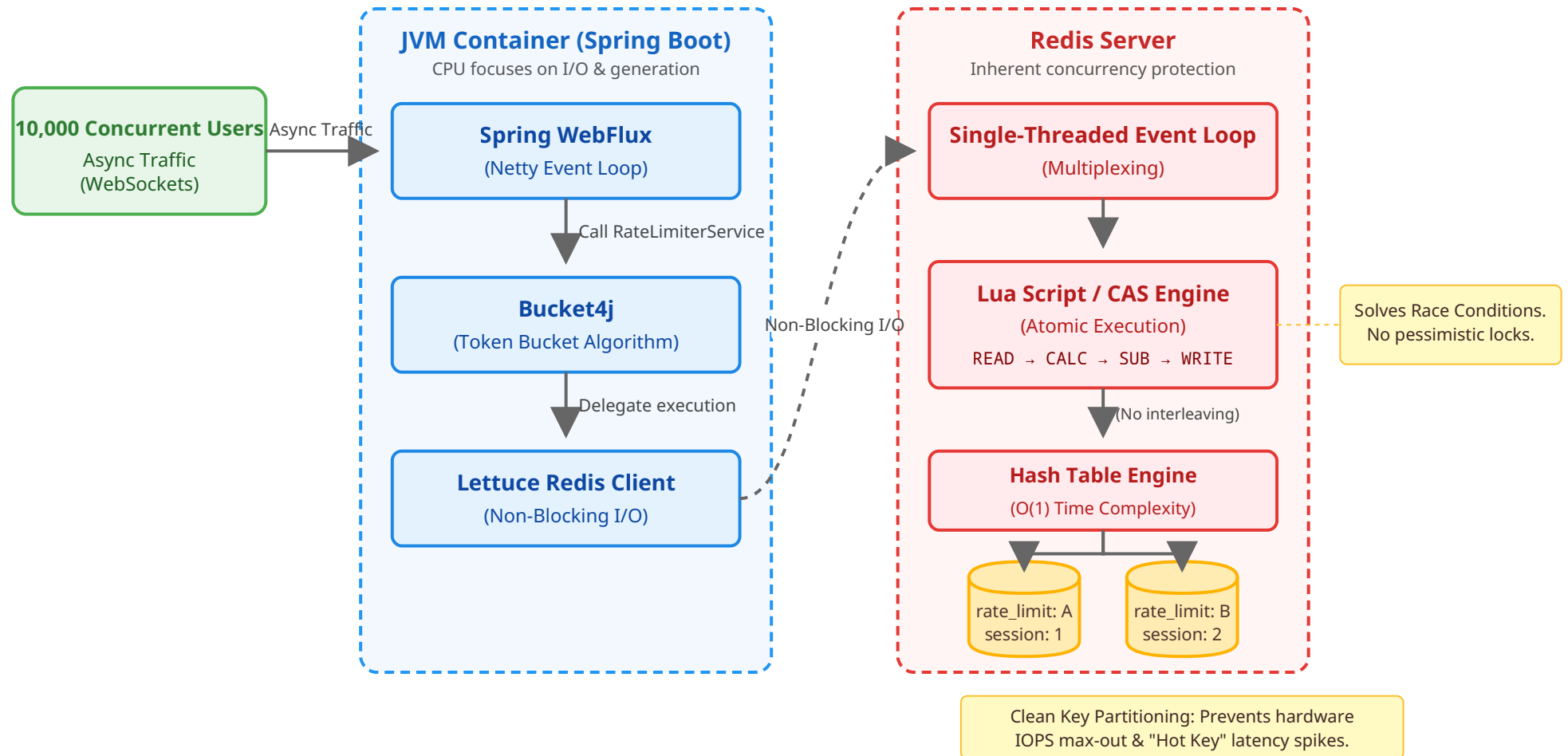


Sentinel Rate Limiting Architecture

System Block Diagram & Component Breakdown



1. Client Layer

10,000 Concurrent Users: The architecture supports massive scale via non-blocking asynchronous traffic streams.

Protocol: Data is transmitted using **WebSockets** to maintain low-latency, persistent connections without standard HTTP overhead.

2. Application Layer (Spring Boot)

Resource Allocation: The JVM focuses purely on I/O multiplexing and generation.

- **Spring WebFlux:** Uses the Netty event loop to handle concurrent connections asynchronously.
- **Bucket4j:** Encapsulates the Token Bucket mathematical algorithm for distributed rate limiting.
- **Lettuce Redis Client:** Dispatches atomic commands to the data store using non-blocking I/O.

3. Data Store (Redis)

Inherent Protection: Single-threaded execution natively protects against race conditions without pessimistic locks.

- **Event Loop:** Sequentially processes incoming commands via multiplexing.
- **CAS Engine (Lua):** Ensures atomic state transitions (READ → CALC → SUB → WRITE) with absolutely no interleaving.
- **Hash Engine:** Provides $O(1)$ time complexity.
- **Clean Partitioning:** Keys are segmented by tenant and session to prevent IOPS saturation and mitigate "Hot Key" latency spikes.